

Chromedriver 卡死问题排查与修复记录

日期：2026-08-26 环境：Windows 10, Python 3.11.15, Selenium 4.47.0, Chrome 151.0.7922.170 结论：问题出在"chromedriver 下载源不可达"，修复是"改用本地驱动 + 镜像下载，并写入源码"。

一、报错现象

执行 `uv run fudan-test` 后：

- 1. `uv` 环境建立成功（依赖安装正常）。
- 2. 程序直接卡住，**Chrome 窗口没有弹出**。
- 3. 无法用 `Ctrl+C` 中断进程，只能强杀终端。

二、根因分析

1. 表层原因：Selenium Manager 联网下载驱动时挂死

Selenium 4.x 默认由 **Selenium Manager** 自动解析并下载匹配的 chromedriver：

```
webdriver.Chrome()  
└─ Selenium Manager 请求 googlechromelabs.github.io 获取版本列表  
   └─ 从 storage.googleapis.com 下载 chromedriver-win64.zip  
      └─ 解压到 ~/.cache/selenium/ 并返回路径
```

实测连通性（关键证据）：

目标地址	作用	结果
googlechromelabs.github.io	版本列表	间歇可达（HTTP 200，约 1.4s）
storage.googleapis.com	驱动二进制 CDN	完全不可达 （HTTP 000，20s 超时）

驱动二进制所在的 Google 存储在**国内网络下连接挂起**（不是快速失败，而是卡在连接阶段），导致 `webdriver.Chrome()` 无限阻塞，Python 进程收不到 `Ctrl+C`，Chrome 自然不会弹出。

注意：不是"所有外网都连不上"——GitHub、npmmirror 都可达，连不上的是 **Google 托管的下载源**。

2. 深层原因：本地 Selenium Manager 缓存损坏

`~/.cache/selenium/` 的元数据 `se-metadata.json` 声称"已缓存 chromedriver 151.0.7922.138"，但实际 `chromedriver/win64/` 目录下**没有任何驱动文件**（只有空目录）。

于是 Selenium Manager 陷入两难：

- 缓存元数据说"驱动在"，不应该重新下载；
- 但文件其实不存在，也不能直接用。

最终只能反复尝试联网解析，在网络不可达时表现为无限挂起。

3. 直接证据（受控复现）

用带硬超时的脚本复现，得到明确报错而不是挂死：

```
selenium.common.exceptions.WebDriverException:
Unsuccessful command executed: ...\\selenium-manager.exe --browser chrome ... ;
code: 65
{'code': 65, 'message': 'error sending request for url
(https://googlechromelabs.github.io/chrome-for-testing/known-good-versions-with-
downloads.json)',
'driver_path': '', 'browser_path': ''}
```

code: 65 即 Selenium Manager 的网络错误码。

4. 为什么 Ctrl+C 无效？

`webdriver.Chrome()` 阻塞在 Selenium Manager 子进程的网络请求上，该请求卡在 TCP 连接/ TLS 握手阶段。Python 主线程深陷在原生阻塞调用里，**SIGINT** (Ctrl+C) 无法及时被处理成 **KeyboardInterrupt**，所以表现为“杀不掉”。

三、排查过程（按时间顺序）

1. **确认环境基线**：Chrome 已装在标准路径 (151.0.7922.170)；`tasklist` 无残留的 chrome/chromedriver 进程；`asset/` 下无 `cookies.txt` → 走的是手动登录分支。
2. **发现 `.runtime/chrome-profile` 为空**：正常情况下 Chrome 用该目录启动后会立即写入 `Preferences`、`Local State` 等文件；空目录说明 **Chrome 从未真正启动成功**，卡点在 driver 阶段而非页面加载。
3. **检查 Selenium Manager 缓存，发现损坏**：`se-metadata.json` 声称已缓存 `151.0.7922.138`，但 `chromedriver/win64/151.0.7922.138/` 为空。
4. **带硬超时复现，拿到明确错误**：用 `subprocess` + 75s 超时包裹 `webdriver.Chrome()`，得到上面的 **code: 65** 网络报错。
5. **连通性测试定位到具体地址**：`storage.googleapis.com` 完全不可达 → 定位根因；`npmmirror` 的 `chrome-for-testing` 镜像 `https://registry.npmirror.com/-/binary/chrome-for-testing/` 返回 302（可用）。
6. **读 Selenium 源码确认绕过方案**：`driver_finder.py` 的 `_binary_paths()`——只要 `Service` 里指定了有效的 `executable_path`，就完全跳过 **Selenium Manager**，不再发起任何网络请求。

四、修复方案（最终实现）

原则：优先本地，其次镜像，**Google 源**只做最后兜底。

原因：Google 源的失败模式是“挂起”而不是“快速报错”，所以**绝不能先试 Google**。把可用的、带超时的镜像下载放前面，才能保证永远不卡死。

改动范围

只改了一个文件：`cookie_engine.py` (+134 行)。其他源码、`pyproject.toml`、`README` 均未改动。

新增常量

```
DRIVER_PATH = Path(__file__).resolve().parent / ".runtime" / "chromedriver.exe"
KNOWN_GOOD_URL = (
    "https://googlechromelabs.github.io/chrome-for-testing/known-good-versions-with-downloads.json"
)
MIRROR_TEMPLATE = (
    "https://registry.npmirror.com/-/binary/chrome-for-testing/{version}/win64/chromedriver-win64.zip"
)
```

新增辅助函数

函数	作用
<code>_chrome_exe()</code>	定位 Chrome 安装路径（3 个常见位置逐一探测）
<code>_chrome_major()</code>	读取本机 Chrome 主版本号（PowerShell 查询 exe 版本）
<code>_driver_matches(driver_path, major)</code>	检查本地驱动版本是否与 Chrome 主版本匹配
<code>_download_matching_driver(major)</code>	从 npmmirror 镜像下载匹配的驱动， 所有网络请求带超时 ，失败返回 <code>None</code>
<code>_resolve_driver_path()</code>	按"本地 → 镜像 → 放弃"顺序解析驱动路径

核心逻辑 `_resolve_driver_path()`

1. 读 Chrome 主版本号 `major`
2. 若 `major` 读不到 → 有本地驱动就用，否则放弃（走 Selenium Manager）
3. 本地驱动存在且版本匹配 → 直接返回（零网络，最快）
4. 本地驱动缺失 / 版本过期 → 提示并自动从镜像下载

`create_driver()` 改动

```
service = None
driver_path = _resolve_driver_path()
if driver_path:
    service = webdriver.ChromeService(executable_path=driver_path)
browser = webdriver.Chrome(options=options, service=service)
```

- 拿到本地/镜像驱动 → 显式传给 `ChromeService` → **Selenium Manager 被完全跳过**，不再联网。
- 拿不到（镜像也失败）→ `service=None` → 回退 Selenium Manager 兜底。

自动刷新

Chrome 会自动更新 (151 → 152 → ...), 本地驱动会失配。由于 `_resolve_driver_path()` 每次运行都会比对 Chrome 主版本号, **失配时自动重新从镜像下载**, 无需手动干预。 若想手动强制刷新, 删除 `.runtime/chromedriver.exe` 即可。

五、验证结果

验证项	结果
显式本地驱动创建 driver	✅ 1.0s 创建成功, 页面正常加载
移除本地驱动后自动从镜像下载	✅ 输出 <code>Downloading chromedriver 151.0.7922.138 from mirror... → Installed driver at ...</code>
Chrome 正常弹出	✅ 窗口可见 (运行期间被手动关闭, 正好证明能弹窗)
单元测试	✅ <code>python -m unittest discover -s tests</code> → 6/6 通过
环境残留	✅ 无孤儿进程、无临时文件

六、使用说明

```
# 正常使用 (Chrome 弹出, 手动完成复旦登录)
uv run fudan-test

# 其他用户 clone 后直接可用:
# 首次运行若 .runtime/chromedriver.exe 不存在, 会自动从镜像下载, 无需任何手动步骤。
```

七、涉及文件

文件	改动
<code>cookie_engine.py</code>	+134 行: 新增驱动解析/下载逻辑, <code>create_driver</code> 优先本地
<code>.runtime/chromedriver.exe</code>	运行产物 (已被 <code>.gitignore</code> 忽略, 不提交)